

# Sistema de Planeamento de Horários

Carolina Cruz n.º 50475  
Constança Costa n.º 50541

Orientador: Paulo Pereira

Relatório do projeto realizado no âmbito de Projecto e Seminário  
Licenciatura em Engenharia Informática e de Computadores

Junho de 2026



# INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA

## Sistema de Planeamento de Horários

50475 Carolina Cruz

---

50541 Constança Costa

---

Orientador: Paulo Pereira

---

Relatório do projeto realizado no âmbito de Projecto e Seminário  
Licenciatura em Engenharia Informática e de Computadores

Junho de 2026



# Resumo

O presente relatório descreve o desenvolvimento da versão beta do projeto *Sistema de Planeamento de Horários*, realizado no âmbito da unidade curricular de Projecto e Seminário da Licenciatura em Engenharia Informática e de Computadores. O objetivo principal do projeto consiste na criação de uma solução que apoie a gestão e criação automática de horários entre professores e alunos, tendo em conta disponibilidades semanais, restrições de agendamento e associações entre utilizadores.

A solução desenvolvida assenta numa aplicação Android implementada em Kotlin com Jetpack Compose e num backend em Ktor com API REST, suportado por persistência remota em PostgreSQL. Ao longo desta fase foram implementadas funcionalidades centrais como:

- o registo e consulta de professores e alunos,
- a definição de disponibilidades e restrições de horário,
- a associação entre alunos e professores,
- a criação automática de propostas de horário com base nas disponibilidades e restrições definidas.

A versão beta permitiu validar a arquitetura global da solução, a comunicação entre frontend e backend e o funcionamento dos fluxos principais do sistema. Foram igualmente realizados testes unitários, testes funcionais da aplicação e validação dos endpoints da API com recurso ao Postman. Os resultados obtidos demonstram a viabilidade técnica da solução e estabelecem uma base consistente para a continuação do desenvolvimento até à versão final.



# Utilização de Inteligência Artificial

Neste trabalho foram utilizadas ferramentas de inteligência artificial generativa como apoio à análise, revisão técnica, esclarecimento de dúvidas e apoio à redação de documentação. Estas ferramentas foram utilizadas como suporte ao desenvolvimento, não substituindo a implementação, validação e compreensão técnica por parte das autoras.





# Índice

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| <b>1</b> | <b>Introdução</b>                                     | <b>1</b>  |
| 1.1      | Motivação e Objetivos . . . . .                       | 1         |
| 1.2      | Visão Geral da Solução . . . . .                      | 2         |
| 1.3      | Organização do Documento . . . . .                    | 3         |
| <b>2</b> | <b>Enquadramento</b>                                  | <b>5</b>  |
| 2.1      | Problema e Contexto de Aplicação . . . . .            | 5         |
| 2.2      | Trabalho Relacionado e Soluções Semelhantes . . . . . | 6         |
| 2.3      | Justificação da Solução Proposta . . . . .            | 6         |
| <b>3</b> | <b>Arquitetura da Solução</b>                         | <b>9</b>  |
| 3.1      | Visão Global da Arquitetura . . . . .                 | 9         |
| 3.2      | Arquitetura do Frontend . . . . .                     | 10        |
| 3.3      | Arquitetura do Backend . . . . .                      | 10        |
| 3.4      | Persistência de Dados . . . . .                       | 11        |
| 3.5      | Comunicação entre Frontend e Backend . . . . .        | 11        |
| 3.6      | Considerações sobre a Arquitetura Atual . . . . .     | 12        |
| <b>4</b> | <b>Implementação do Sistema</b>                       | <b>13</b> |
| 4.1      | Organização Funcional da Aplicação . . . . .          | 13        |
| 4.2      | Gestão de Utilizadores e Perfis . . . . .             | 13        |
| 4.3      | Gestão de Disponibilidades . . . . .                  | 14        |
| 4.4      | Gestão de Restrições . . . . .                        | 14        |
| 4.5      | Associação entre Alunos e Professores . . . . .       | 15        |
| 4.6      | Geração de Propostas de Horário . . . . .             | 15        |
| 4.7      | Visualização do Horário Gerado . . . . .              | 16        |
| 4.8      | Integração Progressiva com o Backend . . . . .        | 16        |
| <b>5</b> | <b>Backend e API</b>                                  | <b>19</b> |
| 5.1      | Escolha Tecnológica . . . . .                         | 19        |
| 5.2      | Estrutura do Backend . . . . .                        | 19        |

|          |                                                     |           |
|----------|-----------------------------------------------------|-----------|
| 5.3      | Modelo de Persistência no Backend . . . . .         | 20        |
| 5.4      | API REST Implementada . . . . .                     | 20        |
| 5.5      | Geração de Horários no Backend . . . . .            | 21        |
| 5.6      | Validação com Postman . . . . .                     | 22        |
| 5.7      | Integração com a Aplicação Android . . . . .        | 22        |
| 5.8      | Síntese do Estado Atual do Backend . . . . .        | 22        |
| <b>6</b> | <b>Testes e Validação</b>                           | <b>25</b> |
| 6.1      | Testes Unitários . . . . .                          | 25        |
| 6.2      | Validação do Backend com Postman . . . . .          | 26        |
| 6.3      | Testes Funcionais da Aplicação Android . . . . .    | 27        |
| 6.4      | Validação da Geração de Horários . . . . .          | 27        |
| 6.5      | Integração e Validação Progressiva . . . . .        | 28        |
| 6.6      | Síntese dos Resultados Obtidos . . . . .            | 28        |
| <b>7</b> | <b>Estado Atual da Beta e Próximos Passos</b>       | <b>29</b> |
| 7.1      | Funcionalidades Concluídas na Versão Beta . . . . . | 29        |
| 7.2      | Estado da Integração do Sistema . . . . .           | 30        |
| 7.3      | Limitações Atuais . . . . .                         | 30        |
| 7.4      | Próximos Passos . . . . .                           | 31        |
| 7.5      | Balanco da Versão Beta . . . . .                    | 31        |
|          | <b>Referências</b>                                  | <b>33</b> |

# Capítulo 1

## Introdução

A organização de horários entre professores e alunos é uma tarefa que, em muitos contextos, continua a ser realizada de forma manual, recorrendo a trocas sucessivas de mensagens, folhas de cálculo ou registos informais. Este processo tende a ser demorado, pouco flexível e propenso a erros, especialmente quando existem múltiplos alunos, disponibilidades distintas e restrições específicas que precisam de ser respeitadas. A dificuldade aumenta quando se pretende encontrar uma distribuição equilibrada de sessões, compatível com a disponibilidade do professor e com as preferências ou limitações dos alunos.

A motivação para este projeto surgiu de uma experiência concreta das autoras no contexto de um centro de estudos, onde ambas desempenham funções de explicadoras. Neste contexto, a gestão dos horários dos alunos era realizada manualmente e exigia reajustes frequentes sempre que entrava um novo aluno ou surgiam alterações de disponibilidade. Na prática, isso implicava refazer sucessivamente os horários já definidos, tentar encontrar novos encaixes e, muitas vezes, lidar com a dificuldade de conciliar todos os alunos de forma equilibrada e sustentável. Esta experiência permitiu identificar de forma clara a necessidade de uma solução que apoiasse este processo de forma mais eficiente e estruturada.

Neste contexto, surgiu a proposta do projeto *Sistema de Planeamento de Horários*, cujo objetivo consiste no desenvolvimento de uma solução capaz de apoiar a gestão e criação automática de horários entre professores e alunos. A solução permite recolher disponibilidades, definir restrições de agendamento e gerar propostas de horário de forma estruturada, reduzindo o esforço manual associado a este processo e tornando a gestão de sessões mais clara e eficiente.

### 1.1 Motivação e Objetivos

A motivação principal para o desenvolvimento deste projeto prende-se com a necessidade de criar uma ferramenta capaz de simplificar a coordenação de horários em cenários onde um professor trabalha com vários alunos e necessita de conciliar diferentes disponibilidades ao longo da semana. Em vez de depender exclusivamente de planeamento manual, a solução

proposta procura automatizar parte desse trabalho, permitindo que os dados relevantes sejam recolhidos de forma consistente e utilizados para construir propostas de horário com base em regras bem definidas.

O objetivo central do projeto consiste, assim, em disponibilizar um sistema que permita:

- registar professores e alunos;
- associar alunos a um professor;
- recolher as disponibilidades semanais de professores e alunos;
- definir restrições relevantes para o agendamento;
- criar automaticamente propostas de horário;
- apresentar o horário resultante de forma legível, organizada, dinamica e facilitada.

Numa perspetiva mais ampla, o projeto pretende também servir como base para a evolução de funcionalidades futuras, como integração com sistemas externos, persistência remota mais completa e eventual suporte a extração automática de informação a partir de documentos ou imagens.

## 1.2 Visão Geral da Solução

A solução desenvolvida assenta numa arquitetura composta por uma aplicação Android, responsável pela interação com o utilizador, e por um backend com API REST, responsável pela persistência e disponibilização de dados centrais do sistema. No frontend foi utilizada a linguagem Kotlin com Jetpack Compose para a construção da interface, enquanto no backend foi adotado Ktor para a implementação da API, em conjunto com PostgreSQL para persistência de dados.

A aplicação distingue dois tipos principais de utilizador através de uma classe "Owner Type" desenvolvida por nós: professor e aluno. Cada um destes perfis tem acesso a um conjunto próprio de funcionalidades. O professor pode definir a sua disponibilidade, configurar restrições associadas ao planeamento e consultar os alunos que lhe estão associados. O aluno pode escolher um professor e definir a sua própria disponibilidade. A partir destes dados, o sistema pode criar uma proposta de horário semanal compatível com as disponibilidades registadas e com as restrições configuradas. Por sua vez o professor irá avaliar o resultado final e, caso seja necessário, poderá requisitar uma nova proposta de horário. Ao ser aceite a proposta de horário, o sistema irá registá-lo no Google Calendar do professor, utilizando a API do Google Calendar, e irá disponibilizar o horário específico de cada aluno no seu email pessoal.

Na versão beta do projeto, foi dada especial atenção à validação dos fluxos principais do sistema, à integração entre frontend e backend, e à implementação de uma primeira

abordagem para a criação automática de horários. Esta fase permite demonstrar a viabilidade técnica da solução e serve como base para melhorias e refinamentos futuros.

### **1.3 Organização do Documento**

O presente relatório encontra-se organizado da seguinte forma. No Capítulo 2 é apresentado o enquadramento do problema, a motivação do projeto e o contexto em que a solução se insere. No Capítulo 3 é descrita a organização global do sistema, incluindo os principais componentes e a sua interação. Seguidamente, são apresentados os detalhes de implementação mais relevantes, tanto ao nível do frontend como do backend, bem como os principais aspetos da API e da persistência de dados. Posteriormente, é descrita a estratégia de testes e validação aplicada ao sistema. Por fim, é feito um balanço do estado atual da versão beta, identificando as funcionalidades já concluídas, as limitações atuais e os próximos passos previstos para a evolução do projeto.



## Capítulo 2

# Enquadramento

O problema da organização de horários encontra-se presente em múltiplos contextos, nomeadamente em ambientes educativos, centros de formação e serviços em que um responsável necessita de gerir sessões com vários participantes. Nestes cenários, a compatibilização de disponibilidades e restrições é frequentemente um processo complexo, sobretudo quando existe necessidade de acomodar vários alunos, respeitar limites temporais e assegurar uma distribuição adequada das sessões ao longo da semana.

No caso particular deste projeto, o foco recai sobre a construção de uma solução orientada à relação entre professor e alunos, em que o professor define a sua disponibilidade e restrições, os alunos indicam os seus períodos disponíveis e o sistema cria automaticamente propostas de horário compatíveis com a informação recolhida. Este enquadramento combina aspetos de gestão de dados, interação utilizador-sistema e lógica de planeamento, tornando o problema simultaneamente prático e tecnicamente relevante.

### 2.1 Problema e Contexto de Aplicação

A construção manual de horários tende a envolver sucessivas trocas de informação entre os intervenientes, registos informais e verificação manual de compatibilidades. Este método, embora viável em contextos muito reduzidos, torna-se pouco eficiente quando o número de alunos aumenta ou quando as disponibilidades variam de forma significativa. Além disso, a gestão manual não facilita a aplicação consistente de regras como a duração das sessões, o número máximo de alunos por sessão ou o número máximo de sessões permitidas por dia.

O projeto desenvolvido procura responder a este problema através de uma solução digital que apoia a recolha e organização de informação relevante para o agendamento. Em vez de depender exclusivamente da intervenção manual do utilizador para encontrar combinações possíveis, o sistema recolhe dados estruturados e utiliza-os para construir propostas de horário automaticamente. Esta abordagem permite reduzir o esforço necessário para planear sessões, diminuir erros de compatibilidade e tornar mais clara a gestão do tempo disponível do professor.

O contexto de aplicação da solução é suficientemente abrangente para poder ser adaptado a vários cenários de ensino ou acompanhamento individual e em grupo. Embora a implementação atual esteja focada numa relação entre professor e alunos, o modelo adotado poderá futuramente ser estendido a outros contextos em que exista necessidade de coordenar horários entre um responsável e vários participantes.

## 2.2 Trabalho Relacionado e Soluções Semelhantes

O problema abordado neste projeto insere-se na área mais ampla de *scheduling* e planeamento de recursos, em que se procura distribuir entidades, atividades e intervalos temporais de forma compatível com um conjunto de restrições. Em termos conceptuais, trata-se de um problema em que é necessário representar disponibilidades, aplicar regras de negócio e encontrar combinações válidas entre vários intervenientes. Este tipo de problema surge em várias áreas, como planeamento académico, gestão de turnos, marcação de reuniões e organização de sessões de formação.

Existem atualmente diversas ferramentas digitais que apoiam a marcação de horários, sobretudo em cenários de agendamento individual. Muitas dessas soluções permitem selecionar intervalos disponíveis, marcar sessões e enviar confirmações aos participantes. Contudo, nem sempre oferecem mecanismos ajustados a cenários em que um professor trabalha com vários alunos, em que é necessário cruzar múltiplas disponibilidades e respeitar restrições específicas associadas à organização de sessões.

Em paralelo, aplicações de calendário amplamente utilizadas, como soluções de gestão de agenda e serviços de calendário online, facilitam a visualização e o registo de eventos, mas não resolvem diretamente o problema de construir automaticamente um horário com base em informação distribuída por vários utilizadores e restrições próprias do domínio. Assim, embora sejam úteis como ferramentas complementares, não substituem uma solução pensada especificamente para este contexto.

A solução desenvolvida neste projeto posiciona-se, por isso, como uma plataforma orientada a um caso de uso mais específico, combinando funcionalidades de gestão de utilizadores, recolha de disponibilidades, definição de restrições e criação de propostas de horário. Em vez de atuar apenas como repositório de eventos, o sistema procura apoiar de forma ativa o processo de planeamento, servindo como base para uma automatização progressiva da criação de horários.

## 2.3 Justificação da Solução Proposta

A escolha de desenvolver uma solução própria justifica-se pela necessidade de responder a requisitos concretos que não são completamente satisfeitos por ferramentas genéricas de calendário ou por sistemas simples de marcação. O projeto pretende integrar, numa única



solução, vários elementos que normalmente surgem dispersos:

- gestão de utilizadores,
- associação entre alunos e professor,
- registo estruturado de disponibilidades,
- configuração de restrições,
- criação automática de propostas de horário.

Além disso, o desenvolvimento do sistema permite explorar aspetos técnicos relevantes do ponto de vista da engenharia de software, nomeadamente a construção de uma aplicação Android, o desenvolvimento de uma API REST, a persistência local e remota de dados, e a implementação de lógica de planeamento baseada em restrições. Desta forma, o projeto não se limita a resolver um problema prático, mas constitui também uma oportunidade para consolidar competências técnicas adquiridas ao longo do curso e novas competências relacionadas com a utilização de LLM's e bibliotecas externas.

A versão beta do projeto procura, assim, demonstrar a viabilidade desta abordagem, validando os fluxos principais da solução e preparando o caminho para funcionalidades futuras, como reforço da integração entre componentes, melhoria do algoritmo de planeamento e eventual suporte a novos tipos de entrada de dados.



## Capítulo 3

# Arquitetura da Solução

A solução desenvolvida assenta numa arquitetura composta por dois blocos principais: uma aplicação móvel Android, responsável pela interação com os utilizadores, e um backend com API REST, responsável pela persistência central de dados e pela execução da lógica associada à gestão de informação do sistema. Esta separação permite distribuir responsabilidades de forma clara entre os componentes, simplificando a evolução da solução e favorecendo a manutenção do código.

De forma geral, a aplicação Android permite ao utilizador interagir com o sistema de acordo com o seu perfil, nomeadamente professor ou aluno, enquanto o backend disponibiliza os serviços necessários para registo e consulta de informação. A comunicação entre ambos é realizada através de pedidos HTTP a uma API REST, permitindo que o frontend envie dados estruturados e obtenha respostas consistentes para os vários fluxos funcionais da aplicação.

### 3.1 Visão Global da Arquitetura

A arquitetura adotada segue uma abordagem em camadas, onde cada componente desempenha uma função específica. No frontend, a interface foi construída com Jetpack Compose, recorrendo a *ViewModels* e repositórios para organizar o estado da aplicação e a lógica de acesso a dados. No backend, a API foi implementada em Ktor, com persistência em PostgreSQL e uma camada de acesso à base de dados suportada por Exposed e HikariCP.

Em termos funcionais, a arquitetura pode ser vista como composta pelos seguintes elementos:

- aplicação Android, que apresenta os ecrãs e gere a interação com o utilizador;
- camada de lógica no frontend, responsável por coordenar o estado da interface e os fluxos de utilização;
- API REST no backend, que centraliza as operações de leitura, escrita e consulta de dados;
- base de dados PostgreSQL, que armazena a informação persistente do sistema;

- armazenamento local com Room, utilizado como suporte local no dispositivo Android.

Esta organização permite que a aplicação continue a evoluir de forma modular, com fronteiras relativamente claras entre apresentação, lógica de negócio, comunicação remota e persistência.

## 3.2 Arquitetura do Frontend

O frontend da solução foi desenvolvido em Kotlin para Android, utilizando Jetpack Compose como tecnologia principal para construção da interface. Esta escolha permite uma abordagem declarativa ao desenho dos ecrãs, facilitando a gestão de estados e a criação de componentes reutilizáveis.

A aplicação encontra-se organizada em várias camadas, com destaque para:

- *screens*, responsáveis pela apresentação visual das funcionalidades;
- *ViewModels*, responsáveis pela gestão do estado e coordenação da lógica de apresentação;
- *repositories*, responsáveis pelo acesso a dados locais e remotos;
- entidades locais em Room, que suportam persistência local no dispositivo.

Do ponto de vista funcional, a aplicação distingue dois perfis de utilizador: professor e aluno. Cada perfil possui ecrãs e ações específicas. O professor pode gerir a sua disponibilidade, definir restrições e consultar os alunos associados. O aluno pode escolher um professor e registar a sua disponibilidade. Esta separação por papéis foi integrada tanto na navegação da aplicação como na própria lógica dos *ViewModels*.

## 3.3 Arquitetura do Backend

O backend foi desenvolvido em Kotlin com Ktor, funcionando como servidor de aplicação e ponto central de integração de dados. A API implementada segue o estilo REST e expõe endpoints para operações como:

- criação e consulta de professores e alunos;
- associação de alunos a professores;
- registo e consulta de disponibilidades;
- registo e consulta de restrições;
- geração de propostas de horário.

A estrutura interna do backend encontra-se organizada em componentes como:

- definição de rotas, responsável por mapear pedidos HTTP para operações concretas;
- modelos e DTOs, que representam os dados trocados entre cliente e servidor;
- serviços, que encapsulam partes relevantes da lógica do sistema;
- acesso à base de dados, com apoio de Exposed para modelação e manipulação de tabelas.

A escolha de Ktor permitiu construir um backend leve, relativamente simples de estruturar e bem alinhado com o restante ecossistema Kotlin do projeto. Esta solução revelou-se adequada para suportar o conjunto de operações necessárias à versão beta e para validar a integração com a aplicação móvel.

### 3.4 Persistência de Dados

A persistência do sistema está dividida em dois níveis complementares. No backend, os dados principais são armazenados em PostgreSQL, funcionando esta base de dados como repositório central do sistema. No frontend, foi utilizado Room para persistência local, permitindo manter dados no dispositivo e suportar alguns fluxos locais ou híbridos durante a evolução da aplicação.

Na base de dados central são armazenadas entidades essenciais ao domínio do problema, tais como:

- professores;
- alunos;
- disponibilidades;
- restrições.

A utilização de Room no frontend permite preservar alguma informação localmente, o que facilita o funcionamento de certas operações e reduz dependência imediata da comunicação remota em todos os momentos. No entanto, o objetivo global da arquitetura é que a API e a base de dados remota assumam progressivamente o papel de fonte principal de verdade do sistema.

### 3.5 Comunicação entre Frontend e Backend

A comunicação entre a aplicação Android e o backend é feita através de pedidos HTTP para a API REST, utilizando o endereço local do servidor durante o desenvolvimento. Os dados trocados entre os componentes são serializados em formato JSON, permitindo uma representação simples e facilmente interpretável por ambas as partes.

Esta comunicação suporta já um conjunto importante de operações, incluindo:

- criação de professores e alunos;
- consulta de listas de utilizadores;
- associação de alunos a professores;
- registo e consulta de disponibilidades;
- registo e consulta de restrições;
- geração de propostas de horário.

A existência desta comunicação remota permite validar o sistema como uma solução efetivamente distribuída, em que a aplicação cliente e o backend colaboram para concretizar os fluxos principais do projeto. A versão beta representa uma etapa importante desta integração, demonstrando que os componentes já comunicam entre si e que a base funcional do sistema se encontra operacional.

### 3.6 Considerações sobre a Arquitetura Atual

A arquitetura adotada foi escolhida tendo em conta a necessidade de construir uma solução progressivamente, validando primeiro os fluxos principais e a integração entre componentes, antes de avançar para otimizações mais profundas. A existência simultânea de persistência local e remota reflete a evolução incremental do projeto, permitindo apoiar o desenvolvimento da aplicação móvel ao mesmo tempo que se consolida a API e a base de dados central.

Embora a arquitetura atual já permita demonstrar de forma consistente o funcionamento da solução, existem ainda aspetos a consolidar, nomeadamente o reforço da sincronização entre dados locais e remotos e a clarificação do papel de cada camada em alguns fluxos específicos. Ainda assim, a base estrutural da solução encontra-se definida e suficientemente sólida para suportar a continuação do desenvolvimento até à versão final.

## Capítulo 4

# Implementação do Sistema

A implementação do sistema foi realizada de forma incremental, começando pela definição dos fluxos principais da aplicação Android e evoluindo depois para a integração com o backend e com a base de dados remota. Esta abordagem permitiu validar gradualmente os principais casos de uso do projeto, garantindo que cada funcionalidade essencial ficava operacional antes da introdução de novos componentes ou maior complexidade.

A solução resultante permite já concretizar um conjunto alargado de operações centrais ao domínio do problema, nomeadamente registo de utilizadores, definição de disponibilidades, gestão de restrições, associação entre alunos e professores e geração de propostas de horário.

### 4.1 Organização Funcional da Aplicação

A aplicação foi concebida em torno de dois perfis principais de utilizador: professor e aluno. A interface e os fluxos disponíveis são ajustados a cada um destes perfis, permitindo apresentar apenas as operações relevantes para cada caso.

A implementação do frontend procurou manter uma separação clara entre:

- apresentação visual da interface;
- estado da aplicação e lógica de interação;
- acesso a dados locais e remotos.

Desta forma, cada ecrã da aplicação está associado a um conjunto de operações específicas, suportadas por *ViewModels* e repositórios que coordenam a lógica necessária à execução dos respetivos fluxos.

### 4.2 Gestão de Utilizadores e Perfis

Uma das primeiras preocupações da implementação foi distinguir corretamente os dois tipos de utilizador suportados pelo sistema. Esta separação é essencial porque professores e alunos desempenham papéis diferentes no problema do planeamento de horários.

Na solução implementada:

- o professor pode definir a sua disponibilidade, configurar restrições e consultar os alunos associados;
- o aluno pode escolher um professor e registrar a sua disponibilidade;
- a navegação da aplicação adapta-se ao perfil autenticado, apresentando diferentes opções no *dashboard*.

A gestão de sessão local foi integrada no frontend, permitindo preservar informação sobre o utilizador autenticado e suportar a adaptação dos fluxos da aplicação ao papel desempenhado. Esta distinção entre perfis foi também refletida no modelo de dados e na lógica dos ecrãs principais.

### 4.3 Gestão de Disponibilidades

A funcionalidade de gestão de disponibilidades representa uma parte fundamental da solução, pois constitui a base sobre a qual são construídas as propostas de horário. Tanto professores como alunos podem registar a sua disponibilidade semanal, indicando os dias e intervalos horários em que se encontram disponíveis.

Numa fase inicial da implementação, a disponibilidade foi modelada de forma mais simples, com um único intervalo comum aos dias selecionados. Contudo, verificou-se que esta abordagem não representava adequadamente situações reais, em que diferentes dias podem ter janelas horárias distintas. Por esse motivo, a funcionalidade foi evoluída para suportar disponibilidade por dia, permitindo que cada dia da semana possua horários próprios.

Esta evolução teve impacto direto na interface e na lógica da aplicação, permitindo:

- selecionar dias individualmente;
- definir horas de início e fim específicas para cada dia;
- persistir e recuperar corretamente essa informação.

A gestão de disponibilidades foi implementada de forma a suportar persistência local e, progressivamente, comunicação com o backend, permitindo que os dados fiquem disponíveis para posterior utilização na geração de horários.

### 4.4 Gestão de Restrições

Para além das disponibilidades, o sistema implementa um conjunto de restrições que orientam a construção do horário do professor. Estas restrições representam regras de negócio relevantes para o problema e são definidas a partir do perfil do professor.

Na implementação atual, o professor pode configurar:



- número máximo de horas diárias;
- duração de cada sessão;
- número máximo de alunos por sessão;
- número máximo de sessões por aluno por dia.

A introdução desta funcionalidade permitiu passar de uma simples compatibilização de horários para um problema de planeamento mais próximo do caso de uso real. As restrições são persistidas e utilizadas posteriormente pelo mecanismo de geração de horários, desempenhando um papel central na validação das sessões propostas.

## 4.5 Associação entre Alunos e Professores

Outro aspeto relevante da implementação foi a definição do modelo de associação entre alunos e professores. A solução adotada considera que cada aluno pode escolher um único professor, enquanto um professor pode ter vários alunos associados.

Esta decisão foi refletida:

- no modelo de dados;
- na lógica do frontend;
- nos endpoints do backend;
- nos ecrãs de seleção e visualização de alunos.

Na aplicação, o aluno pode selecionar um professor a partir da lista de professores disponíveis. Por sua vez, o professor pode consultar os alunos que lhe estão associados. Foi também introduzida a possibilidade de desassociar um aluno, removendo a ligação ao professor sem eliminar o registo do aluno do sistema.

Esta funcionalidade é essencial para garantir que a geração do horário é feita apenas com base nos alunos relevantes para cada professor, tornando o problema mais bem definido e mais próximo do contexto de utilização previsto.

## 4.6 Geração de Propostas de Horário

A **geração** de horários constitui o núcleo funcional do projeto. A implementação atual baseia-se na recolha de disponibilidades do professor e dos seus alunos, na consideração das restrições definidas e na construção de sessões válidas a partir da interseção desses dados.

A lógica de **geração** parte de uma representação em blocos temporais, obtidos a partir dos intervalos de disponibilidade registados. Estes blocos são depois analisados para identificar combinações possíveis entre professor e alunos. O processo de construção das sessões considera, entre outros aspetos:

- compatibilidade de disponibilidade entre professor e alunos;
- limite máximo de alunos por sessão;
- limite de sessões por aluno no mesmo dia;
- limite de carga diária do professor.

A versão beta implementa uma primeira abordagem funcional a este problema, suficiente para demonstrar o comportamento do sistema e validar o fluxo completo de criação de horários. Embora o algoritmo atual ainda possa ser refinado em versões futuras, já permite produzir propostas coerentes para cenários representativos do domínio do projeto.

## 4.7 Visualização do Horário Gerado

Após a criação das sessões, a aplicação apresenta o horário **gerado** de forma organizada e legível. Em vez de mostrar apenas uma lista simples de resultados, foi construída uma visualização semanal agrupada por dia, onde cada sessão aparece com o respetivo intervalo horário e a lista de alunos atribuídos.

Esta representação permite ao utilizador compreender facilmente:

- em que dias existem sessões;
- quais os intervalos temporais ocupados;
- que alunos foram alocados a cada sessão;
- em que dias não existem sessões previstas.

A implementação desta visualização constitui uma componente importante da versão beta, pois transforma o resultado do algoritmo num artefacto diretamente compreensível e utilizável pelo utilizador final.

## 4.8 Integração Progressiva com o Backend

Ao longo da implementação, foi também sendo reforçada a integração entre o frontend e o backend. Algumas operações começaram por ser validadas localmente, recorrendo à persistência em Room, e foram posteriormente adaptadas para consumir a API REST e interagir com o PostgreSQL.

Este processo de integração foi realizado de forma progressiva, permitindo validar cada funcionalidade antes da passagem completa para a componente remota. Como resultado, a versão beta já demonstra um nível significativo de comunicação entre a aplicação Android e o backend, particularmente nos fluxos de utilizadores, disponibilidades, restrições e geração de horário.

A implementação atual representa, assim, uma base funcional robusta sobre a qual será possível continuar a consolidar a solução até à versão final, reduzindo gradualmente a dependência de mecanismos locais auxiliares e reforçando o papel do backend como componente central do sistema.



## Capítulo 5

# Backend e API

O backend do sistema foi desenvolvido com o objetivo de centralizar a persistência remota de dados e disponibilizar uma API capaz de suportar os principais fluxos funcionais da aplicação. Esta componente desempenha um papel essencial na arquitetura da solução, funcionando como ponto de articulação entre a aplicação Android e a base de dados PostgreSQL.

A introdução do backend permitiu ultrapassar as limitações de uma solução exclusivamente local, abrindo caminho para uma estrutura mais próxima de um sistema real distribuído, onde os dados principais deixam de depender apenas do armazenamento existente no dispositivo do utilizador.

### 5.1 Escolha Tecnológica

A implementação do backend foi realizada em Kotlin, utilizando Ktor como framework principal para criação da API REST. Esta escolha revelou-se adequada ao contexto do projeto, sobretudo por permitir manter consistência com a linguagem já utilizada no frontend e por oferecer uma base relativamente simples e leve para a construção do servidor.

A persistência foi suportada por PostgreSQL, escolhido por ser um sistema de gestão de bases de dados relacional robusto, amplamente utilizado e adequado ao armazenamento estruturado das entidades do projeto. Para a interação com a base de dados foi utilizada a biblioteca Exposed, que facilita a definição de tabelas e a execução de operações de acesso a dados em Kotlin. A gestão de conexões foi assegurada por HikariCP, permitindo configurar o acesso à base de dados de forma eficiente e consistente.

No conjunto, estas tecnologias permitiram construir uma base sólida para a versão beta, conciliando simplicidade de desenvolvimento com capacidade de evolução futura.

### 5.2 Estrutura do Backend

O backend foi organizado em vários componentes com responsabilidades distintas, de forma a manter o código compreensível e modular. Entre os principais elementos da sua estrutura destacam-se:

- ficheiro principal de arranque da aplicação, responsável pela inicialização do servidor;
- configuração das rotas da API;
- modelos e DTOs utilizados na troca de informação entre cliente e servidor;
- serviços responsáveis por partes relevantes da lógica do sistema;
- configuração e inicialização da ligação à base de dados;
- definição das tabelas persistidas em PostgreSQL.

Esta organização permite separar as diferentes preocupações do sistema, distinguindo a exposição de endpoints, a representação dos dados, a lógica funcional e o acesso à persistência.

### 5.3 Modelo de Persistência no Backend

A camada de persistência do backend foi concebida para suportar as entidades centrais do domínio do projeto. Na versão beta, foram modeladas estruturas que permitem armazenar:

- professores;
- alunos;
- associação entre alunos e professor;
- disponibilidades;
- restrições associadas ao professor.

A informação é persistida em PostgreSQL, permitindo manter um repositório central do estado do sistema. Esta abordagem representa um passo importante relativamente às fases iniciais do projeto, em que a aplicação dependia fortemente de persistência local. Com a introdução do backend, foi possível começar a transferir a responsabilidade da gestão de dados principais para uma componente remota, mais adequada a uma solução multiutilizador.

No caso das disponibilidades, o backend armazena os intervalos registados por professores e alunos, utilizando essa informação posteriormente para a geração de propostas de horário. No caso das restrições, os dados configurados pelo professor são também persistidos, permitindo que a criação de horários seja baseada em regras efetivamente armazenadas e reutilizáveis.

### 5.4 API REST Implementada

A API desenvolvida no backend foi desenhada para suportar os fluxos principais do sistema. Os endpoints existentes permitem realizar operações de registo, consulta, associação de utilizadores, gestão de disponibilidades, gestão de restrições e criação de horários.

Entre os principais endpoints implementados incluem-se:

- verificação de disponibilidade do servidor;
- criação e consulta de professores;
- criação e consulta de alunos;
- associação e desassociação de alunos a professores;
- consulta de alunos de um professor;
- criação, remoção e consulta de disponibilidades;
- criação e consulta de restrições;
- criação de propostas de horário.

A utilização de uma API REST permitiu estabelecer um contrato claro entre o frontend e o backend, facilitando a integração progressiva da aplicação Android com a componente remota. O formato JSON foi adotado como mecanismo de serialização dos dados, permitindo representar os pedidos e respostas de forma simples e consistente.

## 5.5 Geração de Horários no Backend

Uma das evoluções mais relevantes da versão beta foi a passagem da lógica de geração de horários para o backend. Esta decisão permite centralizar a lógica principal do sistema e reduzir a duplicação de comportamento entre cliente e servidor.

A partir da informação armazenada, o backend:

- obtém a disponibilidade do professor;
- obtém as disponibilidades dos alunos associados;
- carrega as restrições definidas;
- divide os intervalos em blocos temporais adequados;
- gera sessões compatíveis com os dados disponíveis.

Esta lógica permite que a proposta de horário seja construída de forma coerente e reutilizável, garantindo que diferentes clientes possam consumir o mesmo comportamento central sem necessidade de replicar o algoritmo localmente.

## 5.6 Validação com Postman

A validação do backend foi realizada com recurso ao Postman, ferramenta utilizada para testar os endpoints desenvolvidos antes da integração total com o frontend. Este processo foi importante para confirmar que os contratos da API estavam corretos e que as operações principais produziam os resultados esperados.

Durante os testes foram validados fluxos como:

- criação de professores e alunos;
- consulta de listas de utilizadores;
- associação entre aluno e professor;
- registo e consulta de disponibilidades;
- registo e consulta de restrições;
- geração de propostas de horário.

A utilização do Postman permitiu isolar o backend durante o processo de validação, facilitando a identificação de problemas antes da integração direta com a aplicação Android. Este passo revelou-se particularmente útil para testar a coerência das respostas devolvidas pela API e a persistência correta dos dados em PostgreSQL.

## 5.7 Integração com a Aplicação Android

Após a validação inicial dos endpoints, a integração com o frontend foi sendo realizada de forma progressiva. A aplicação Android passou a consumir a API para operações relacionadas com utilizadores, disponibilidades, restrições e, mais recentemente, geração de horários.

Esta integração permitiu aproximar a solução da sua arquitetura final, em que o backend assume o papel de componente central na gestão dos dados e da lógica principal do sistema. Apesar de ainda existirem aspetos híbridos entre persistência local e remota, a evolução observada nesta fase demonstra que a base da comunicação cliente-servidor já se encontra estabelecida e funcional.

## 5.8 Síntese do Estado Atual do Backend

Na versão beta, o backend já se encontra suficientemente estruturado para suportar os principais fluxos do projeto. A API implementada encontra-se funcional, a ligação à base de dados PostgreSQL foi estabelecida com sucesso e os principais endpoints já foram validados de forma independente e em conjunto com a aplicação móvel.



Este estado representa um marco importante no desenvolvimento da solução, pois demonstra que o projeto já não depende apenas de lógica local no frontend. O backend passou a constituir uma parte ativa e central do sistema, permitindo sustentar a evolução futura da solução para uma arquitetura mais consistente, integrada e próxima da versão final pretendida.



## Capítulo 6

# Testes e Validação

A validação do sistema foi realizada de forma progressiva ao longo do desenvolvimento, acompanhando a implementação das diferentes funcionalidades da aplicação e do backend. Esta abordagem permitiu testar isoladamente componentes específicos e, posteriormente, verificar o comportamento integrado da solução. Na versão beta, os testes incidiram sobretudo sobre a lógica de tratamento de disponibilidades, a geração de horários, o funcionamento da API e os fluxos principais da aplicação Android.

A estratégia adotada combinou diferentes níveis de validação, incluindo testes unitários, testes manuais da aplicação e testes aos endpoints da API com recurso ao Postman.

### 6.1 Testes Unitários

Ao nível da lógica local da aplicação, foram implementados testes unitários para componentes diretamente relacionados com a geração de horários e o processamento de disponibilidades. Estes testes tiveram como objetivo garantir que os blocos mais sensíveis da lógica do sistema produziam resultados coerentes e previsíveis.

Entre os componentes testados destacam-se:

- o processador de intervalos temporais, responsável por dividir disponibilidades em blocos utilizáveis pelo algoritmo;
- o gerador de horários, responsável por construir propostas de sessões com base nas disponibilidades e restrições.

No caso do processamento de intervalos temporais, os testes permitiram verificar aspetos como:

- número de blocos gerados para diferentes intervalos;
- preservação correta do dia da semana;
- comportamento em intervalos inválidos ou demasiado curtos;

- suporte a diferentes durações de bloco.

No caso do algoritmo de geração de horários, os testes procuraram validar:

- atribuição correta de alunos disponíveis;
- respeito pelo número máximo de participantes por sessão;
- respeito pelo número máximo de sessões por aluno por dia;
- comportamento em cenários sem alunos ou sem disponibilidade do professor.

Estes testes constituem uma base importante de confiança para o comportamento do sistema, sobretudo nas partes em que a lógica é mais sensível a regras e combinações de dados.

## 6.2 Validação do Backend com Postman

Para a validação do backend foi utilizada a ferramenta Postman, permitindo testar os endpoints da API REST de forma independente da aplicação Android. Esta fase revelou-se importante para confirmar a correção dos contratos HTTP, a estrutura dos pedidos e respostas e a persistência efetiva dos dados em PostgreSQL.

Os testes realizados no Postman abrangeram, entre outros, os seguintes cenários:

- verificação de disponibilidade do servidor;
- criação de professores;
- criação de alunos;
- consulta de listas de professores e alunos;
- associação de alunos a professores;
- consulta dos alunos associados a um professor;
- registo e consulta de disponibilidades;
- registo e consulta de restrições;
- geração de propostas de horário.

Através destes testes foi possível confirmar que a API se encontrava funcional e que os principais fluxos do domínio estavam corretamente representados no backend. Esta validação permitiu também identificar e corrigir inconsistências entre o modelo de dados do frontend e o do backend, contribuindo para uma integração mais robusta entre os dois componentes.

## 6.3 Testes Funcionais da Aplicação Android

Ao nível da aplicação Android, os testes realizados incidiram sobre os principais fluxos funcionais disponíveis para cada tipo de utilizador. O objetivo foi validar o comportamento da interface, a navegação entre ecrãs, a persistência de dados e a integração progressiva com a API.

Entre os fluxos testados destacam-se:

- autenticação e gestão de sessão;
- distinção de comportamento entre professor e aluno;
- seleção de professor por parte do aluno;
- visualização de alunos associados ao professor;
- definição de disponibilidades;
- definição de restrições;
- geração e apresentação do horário.

A validação funcional da aplicação foi importante para garantir que a interface suportava corretamente os comportamentos previstos pelo domínio do problema e que a informação apresentada ao utilizador era consistente com o estado armazenado localmente e, quando aplicável, com a informação obtida do backend.

## 6.4 Validação da Geração de Horários

A geração de horários foi alvo de validação específica, dado tratar-se da funcionalidade central do projeto. Para esse efeito, foram construídos cenários de teste com diferentes combinações de disponibilidades, alunos e restrições, permitindo observar o comportamento do sistema em situações representativas.

Os testes incidiram sobre aspetos como:

- divisão correta dos intervalos de disponibilidade em blocos temporais;
- criação de sessões compatíveis com a disponibilidade do professor;
- alocação de alunos apenas a sessões em que se encontram disponíveis;
- respeito pelos limites de participantes por sessão;
- distribuição de alunos por diferentes blocos quando aplicável.

Através destes testes foi possível confirmar que a versão beta já produz propostas de horário coerentes com as regras definidas e com a informação introduzida no sistema. Embora a abordagem atual ainda possa ser melhorada, os resultados obtidos mostram que a funcionalidade principal do projeto já se encontra operacional.

## 6.5 Integração e Validação Progressiva

Um aspeto relevante da fase de testes foi a validação progressiva da integração entre frontend e backend. Em vez de tentar validar toda a solução apenas no final, as funcionalidades foram sendo testadas à medida que a comunicação remota era introduzida. Esta estratégia permitiu identificar mais cedo inconsistências em endpoints, modelos de dados e fluxos de sincronização.

A coexistência de persistência local e persistência remota exigiu também atenção adicional, uma vez que parte da validação teve de assegurar que os dados obtidos da API se mantinham consistentes com o estado visível na aplicação. Esta abordagem incremental revelou-se útil para consolidar a versão beta e preparar a solução para uma integração mais completa nas fases seguintes do projeto.

## 6.6 Síntese dos Resultados Obtidos

Os testes realizados permitiram concluir que a versão beta do sistema já apresenta um nível de funcionalidade significativo. O backend encontra-se operacional, os principais endpoints foram validados com sucesso, a aplicação Android executa os fluxos centrais do sistema e a geração de horários produz resultados coerentes em cenários representativos.

Apesar de persistirem aspetos a refinar, nomeadamente ao nível da integração total entre componentes e da evolução futura do algoritmo, a validação efetuada demonstra que a solução já consegue suportar os casos de uso mais importantes do projeto. Este resultado constitui um indicador positivo para a continuação do desenvolvimento até à versão final.

## Capítulo 7

# Estado Atual da Beta e Próximos Passos

A versão beta do projeto representa uma fase intermédia de consolidação funcional da solução, em que os principais fluxos do sistema já se encontram implementados e testados, embora ainda existam aspetos a melhorar e integrar totalmente. Esta fase foi essencial para validar a viabilidade técnica da arquitetura adotada, confirmar o funcionamento dos componentes principais e estabelecer uma base sólida para a continuação do desenvolvimento.

A solução já não se encontra limitada a um protótipo exclusivamente local. Com a introdução do backend em Ktor, a integração com PostgreSQL e a utilização de uma API REST funcional, o sistema passou a assumir uma estrutura mais próxima da versão final pretendida.

### 7.1 Funcionalidades Concluídas na Versão Beta

Na fase atual do projeto, já se encontram implementadas as funcionalidades centrais do sistema. Entre os principais elementos já concluídos destacam-se:

- aplicação Android com separação de fluxos entre professor e aluno;
- gestão de sessão e adaptação da interface ao perfil autenticado;
- registo e consulta de professores e alunos;
- associação e desassociação entre alunos e professor;
- definição e persistência de disponibilidades;
- definição e persistência de restrições do professor;
- geração de propostas de horário;
- visualização organizada do horário semanal gerado;

- backend funcional com API REST;
- persistência remota em PostgreSQL;
- validação dos endpoints principais com Postman.

Estes resultados demonstram que a versão beta já suporta os casos de uso mais importantes do projeto, permitindo recolher a informação necessária, processá-la e apresentar ao utilizador uma proposta concreta de horário.

## 7.2 Estado da Integração do Sistema

Um dos principais avanços da versão beta foi a integração progressiva entre frontend e backend. A aplicação Android já comunica com a API para um conjunto importante de operações, nomeadamente na gestão de utilizadores, disponibilidades, restrições e geração de horários. Esta evolução permitiu validar a arquitetura distribuída da solução e reduzir a dependência de mecanismos exclusivamente locais.

Apesar deste progresso, a integração ainda não pode ser considerada totalmente concluída em todos os fluxos. Em alguns pontos, subsistem aspetos híbridos entre persistência local e remota, o que exige atenção adicional para garantir consistência total entre os diferentes componentes. Ainda assim, a base da comunicação entre cliente e servidor encontra-se estabelecida e operacional, constituindo um avanço significativo face às fases iniciais do projeto.

## 7.3 Limitações Atuais

Embora a versão beta demonstre a viabilidade da solução, existem ainda limitações que deverão ser tratadas até à entrega final. Uma dessas limitações prende-se com a necessidade de consolidar plenamente a sincronização entre os dados locais armazenados em Room e os dados persistidos no backend. A coexistência destas duas camadas de persistência, embora útil durante o desenvolvimento incremental, introduz complexidade adicional e requer uma definição mais clara do papel de cada uma na arquitetura final.

Outra limitação atual diz respeito ao algoritmo de geração de horários. A abordagem implementada já é suficiente para validar os fluxos principais e produzir propostas coerentes em cenários representativos, mas poderá ainda ser refinada para responder melhor a casos mais complexos ou a critérios adicionais de qualidade na distribuição das sessões.

Para além disso, existem funcionalidades previstas que ainda não se encontram concluídas, como a exploração de mecanismos adicionais de entrada de dados, a melhoria da cobertura de testes e o reforço da robustez geral do sistema em cenários de utilização mais variados.



## 7.4 Próximos Passos

Tendo em conta o estado atual do projeto, os próximos passos mais relevantes passam por consolidar a solução já implementada e preparar a transição para a versão final. Entre as prioridades identificadas destacam-se:

- concluir a integração total entre frontend e backend;
- reduzir as dependências híbridas entre persistência local e remota;
- reforçar a coerência e sincronização dos dados do sistema;
- melhorar a robustez da geração de horários;
- aprofundar os testes funcionais e de integração;
- melhorar a apresentação final da solução e da demonstração.

Adicionalmente, prevê-se a exploração de funcionalidades mais avançadas, como mecanismos de apoio à extração automática de disponibilidades a partir de inputs externos, caso tal se revele viável dentro do calendário do projeto. Esta linha de evolução poderá acrescentar valor significativo à solução, mas deverá ser enquadrada de forma realista face ao estado atual de desenvolvimento.

## 7.5 Balanço da Versão Beta

De uma forma geral, a versão beta permitiu demonstrar que o projeto já possui uma base funcional consistente e tecnicamente viável. O sistema consegue suportar os fluxos fundamentais do problema, desde a recolha de dados até à criação e apresentação de propostas de horário. A arquitetura adotada mostrou-se adequada à evolução do projeto e os resultados obtidos nesta fase constituem um indicador positivo para a continuação do trabalho.

A partir deste ponto, o foco deixa de estar apenas na implementação de novas funcionalidades e passa também pela consolidação, integração e refinamento do que já foi desenvolvido. Este balanço é particularmente importante, pois permite encarar a versão final não como o início da solução, mas como uma etapa de maturação e aperfeiçoamento de uma base já operacional.



# Referências